LAB 9 - Graph Traversals: BFS and DFS

Data Structures and Algorithms Laboratory

Student Details

Name	Pratyush Deo Singh
Registration Number	25BCE5101
Semester	2nd
Subject	Data Structures and Algorithms
Slot	A2
Faculty Name	Dr. Malini A

Programs

Program 1: Graph BFS using Adjacency List

File Name:

q1_bfs_adjacency_list.c

Program Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    int vertex;
    struct Node* next;
};

struct Node* adjList[MAX];
int visited[MAX];
int queue[MAX];
int front = 0, rear = 0;

struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

void addEdge(int src, int dest) {
    struct Node* newNode = createNode(dest);
```

```
    newNode->next = adjList[src];
    adjList[src] = newNode;

    newNode = createNode(src);
    newNode->next = adjList[dest];
    adjList[dest] = newNode;
}

void enqueue(int v) {
    queue[rear++] = v;
}

int dequeue() {
    return queue[front++];
}

int isEmpty() {
    return front == rear;
}

void bfs(int start) {
    visited[start] = 1;
    enqueue(start);

    printf("BFS Traversal Order: ");

    while (!isEmpty()) {
        int current = dequeue();
        printf("%d ", current);

        struct Node* temp = adjList[current];
        while (temp != NULL) {
            if (!visited[temp->vertex]) {
                visited[temp->vertex] = 1;
                enqueue(temp->vertex);
            }
            temp = temp->next;
        }
    }

    printf("\n");
}

int main() {
    int vertices, edges, src, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < vertices; i++) {
        adjList[i] = NULL;
        visited[i] = 0;
    }

    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }
```

```
    printf("Enter source vertex: ");
    scanf("%d", &src);

    bfs(src);

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

1 4

2 4

Enter source vertex: 0

Expected Output:

BFS Traversal Order: 0 2 1 4 3

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
Enter source vertex: 0
BFS Traversal Order: 0 2 1 4 3
```

Program 2: Graph DFS using Adjacency Matrix

File Name:

q2_dfs_adjacency_matrix.c

Program Code:

```
#include <stdio.h>

#define MAX 100

int adjMatrix[MAX][MAX];
int visited[MAX];
int vertices;

void addEdge(int u, int v) {
    adjMatrix[u][v] = 1;
    adjMatrix[v][u] = 1;
}

void dfs(int src) {
    visited[src] = 1;
    printf("%d ", src);

    for (int i = 0; i < vertices; i++) {
        if (adjMatrix[src][i] == 1 && !visited[i]) {
            dfs(i);
        }
    }
}

int main() {
    int edges, u, v, src;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < vertices; i++) {
        visited[i] = 0;
        for (int j = 0; j < vertices; j++) {
            adjMatrix[i][j] = 0;
        }
    }

    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    printf("DFS Traversal Order: ");
```

```
    dfs(src);
    printf("\n");

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

1 4

2 4

Enter source vertex: 0

Expected Output:

DFS Traversal Order: 0 1 3 4 2

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
Enter source vertex: 0
DFS Traversal Order: 0 1 3 4 2
```

Program 3: DFS using Stack Implementation

File Name:

q3_dfs_stack.c

Program Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    int vertex;
    struct Node* next;
};

struct Node* adjList[MAX];
int visited[MAX];
int stack[MAX];
int top = -1;

struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

void addEdge(int src, int dest) {
    struct Node* newNode = createNode(dest);
    newNode->next = adjList[src];
    adjList[src] = newNode;

    newNode = createNode(src);
    newNode->next = adjList[dest];
    adjList[dest] = newNode;
}

void push(int v) {
    stack[++top] = v;
}

int pop() {
    return stack[top--];
}

int isEmpty() {
    return top == -1;
}

void dfsStack(int start) {
    push(start);

    printf("DFS Traversal Order (Stack): ");

    while (!isEmpty()) {
```

```
        int current = pop();

        if (visited[current])
            continue;

        visited[current] = 1;
        printf("%d ", current);

        struct Node* temp = adjList[current];
        while (temp != NULL) {
            if (!visited[temp->vertex]) {
                push(temp->vertex);
            }
            temp = temp->next;
        }
    }

    printf("\n");
}

int main() {
    int vertices, edges, src, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < vertices; i++) {
        adjList[i] = NULL;
        visited[i] = 0;
    }

    printf("Enter edges (u v):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dfsStack(src);

    return 0;
}
```

Sample Test Input:

Enter number of vertices: 5

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

1 4

2 4

Enter source vertex: 0

Expected Output:

DFS Traversal Order (Stack): 0 2 4 1 3

Output Screenshot:

```
Enter number of vertices: 5
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
Enter source vertex: 0
DFS Traversal Order (Stack): 0 1 3 4 2
```